

PINNs Tutorial

Ahmad Azzeh

May 2026

Contents

1	The Perceptron — One Neuron, One Decision	3
1.1	What a Perceptron Does	3
1.2	Activation Functions	4
1.3	The math behind it - A simple Intro	5
2	Multi Layer Perceptrons	9
2.1	Why are multiple Perceptrons Needed? — The Nonlinearity Problem	9
2.2	Architecture	12
2.3	Why Tanh and Not ReLU or sigmoid for PINNs?	13
2.4	Fourier Feature Embeddings	16
2.4.1	Why Does a Standard MLP Struggle with Sharp Gradients?	16
2.4.2	What Are Fourier Feature Embeddings?	16
3	Backpropagation and Automatic Differentiation	17
3.0.1	The Chain Rule and Intermediate Variables	18
3.0.2	Forward and Reverse Accumulation	19
3.1	Worked Example: Forward Accumulation for $f(x_1, x_2) = x_1x_2 + \sin x_1$	20
3.2	In Each Step: Numerical Storage	22
3.3	The Computation Graph	22
3.4	Worked Example: Two Weights, One Step	23
3.4.1	Worked Example: Two Weights, One Step	23

3.4.2	Example 2 — Sigmoid Neuron	25
3.4.3	Example 3 — Autodiff Exactness: $f(x) = \sin(x) e^x$	26
3.4.4	Backpropagation vs. Autodiff	27
3.4.5	How Backpropagation and Autodiff Work Together	28
4	Nondimensionalisation	28
4.1	Why Nondimensionalisation Matters	28
4.2	The General Procedure	29
4.3	Worked Example: the 1D Transient Heat Equation	29
4.4	Application to the NiTi Wire Problem	30
4.5	What the Network Sees Before and After Scaling	32
4.6	The Three Dimensionless Groups	32

1 The Perceptron — One Neuron, One Decision

The **perceptron** is the fundamental building block of every neural network. It is a single artificial neuron that takes a set of numerical inputs, weighs each one according to its importance, sums the result, and passes it through an **activation function** to produce an output. Before understanding how layers of neurons work together, it is necessary to understand what one neuron does on its own.

1.1 What a Perceptron Does

A perceptron with n inputs $x_1, x_2, \dots, x_n$ computes:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum_{i=1}^n w_ix_i + b \quad (1)$$

$$a = \sigma(z) \quad (2)$$

Where b is the bias term that adjusts the perceptron so it knows when to activate and is independent of any input. The bias shifts the line of the equation either up, down, left or right.

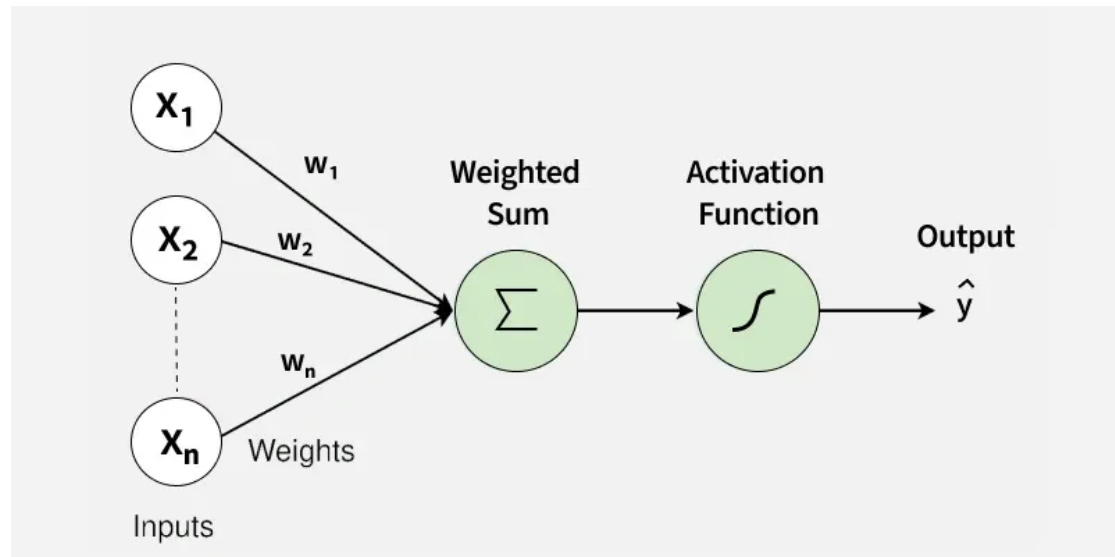


Figure 1: Simple model of a perceptron

Each weight w_i controls how much input x_i influences the result, this is also true for the value of x_i . For example if x_1 is a large value where w_1 is much greater than w_2 , the large w_1 means that the w_1x_i will influence the result much more than w_2x_2 . A value near zero means the perceptron will ignore it.

The sum of each input with its respective weight is called *the weighted sum*. This is then passed onto something called an *activation function* these introduce the non-linearity needed for us to approximate our complex functions.

1.2 Activation Functions

Activation functions are still an on going topic where new techniques and mixes are being used to find the best combination, however for our needs we only need to talk about three activation functions:

- **ReLU:** $\sigma(z) = \max(0, z)$. Computationally cheap but non-differentiable at $z = 0$.
- **Sigmoid:** $\sigma(z) = 1/(1 + e^{-z})$. Smooth and bounded in $(0, 1)$, but suffers from vanishing gradients in deep networks.
- **Tanh:** $\sigma(z) = \tanh(z)$. Smooth, bounded in $(-1, 1)$, zero-centred, and infinitely differentiable.

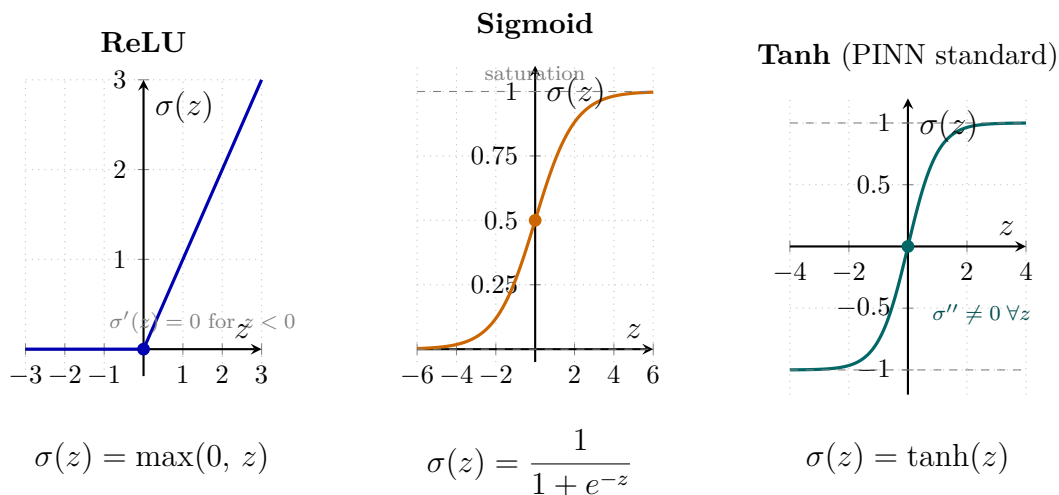


Figure 2: The three activation functions

what is each activation function and when are they used? Activation functions came to be to introduce nonlinearity to the data. Without this nonlinearity the data from the neural networks would just become linear transformers.

From the theory of approximation and based on empirical data, a mathematical model of the biological neuron activation was made and gave rise to the Sigmoid function and the Tanh function (hyperbolic).

The S-shaped of the sigmoid curve approximates a step function and outputs values in (0,1), making it ideal for probabilistic interpretations like binary classification. It was the default activation function in the 1980s and early 1990s.

Hyperbolic tangent (Tanh), outputs values in the range (-1,1), centering the activations around zero. This property accelerates convergence in the early layers of a network by providing stronger gradients compared to the sigmoid function. As a result, tanh became a popular choice of activation function during the late 1990s and early 2000s, especially for neural networks that required more robust training dynamics.

There are advantages and limitations for each activation function.

1.3 The math behind it - A simple Intro

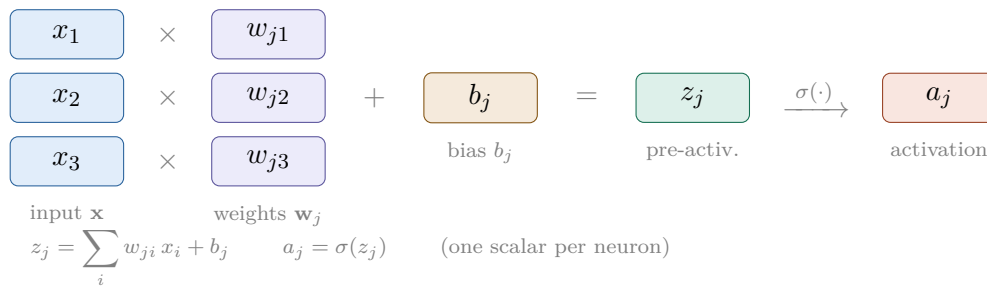


Figure 3: Matrix multiplication of a single perceptron with 3 inputs.

Each input x_i is multiplied by its weight w_{ji} , the products are summed, the bias b_j is added to give the pre-activation z_j , and the activation function σ maps z_j to the neuron's output a_j .

To give a better a better view lets look a what a single perceptron produces.

A perceptron's output is the weighted sum that goes through the activation function. The meaning of the result depends on what activation function you use. For example a sigmoid function will give a probability from 0 to 1, where as tanh will give you a classification from -1 to 1.

To see this better lets use a simple example.

Single Perceptron Approximating $f(x_1, x_2) = \sin(x_1) + x_1x_2$

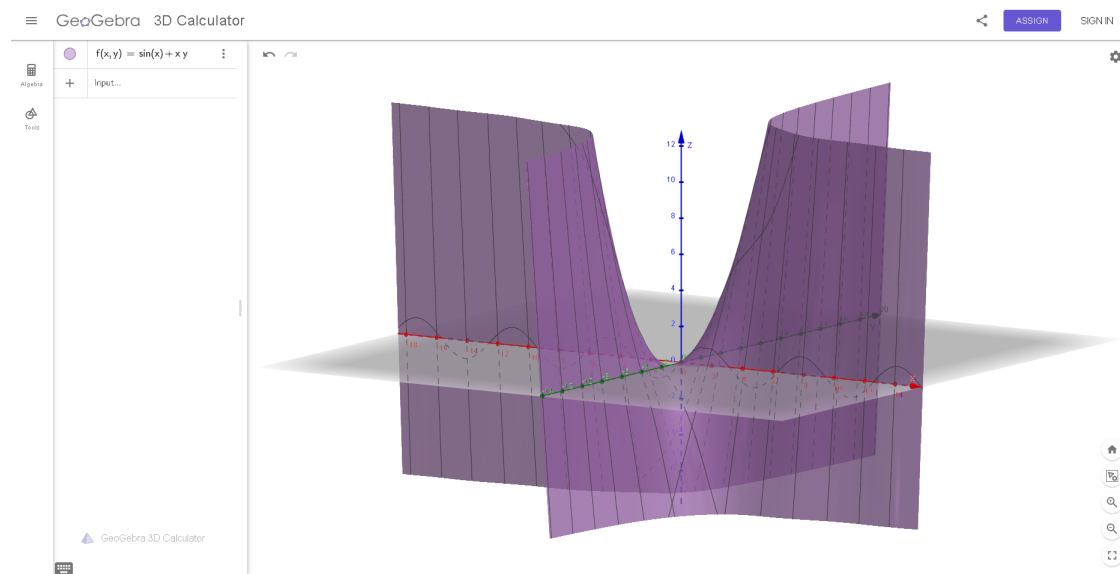


Figure 4: Model of $f(x_1, x_2) = \sin x_1 + x_1x_2$

Consider a single perceptron attempting to approximate the function $f(x_1, x_2) = \sin(x_1) + x_1x_2$ at the point $x_1 = 0.6$, $x_2 = 0.9$. The true value is:

$$\begin{aligned} f(0.6, 0.9) &= \sin(0.6) + (0.6)(0.9) \\ &= 0.565 + 0.540 \\ &= 1.105 \end{aligned}$$

The perceptron has weights $w_1 = -0.8$, $w_2 = 1.5$, and bias $b = -0.3$. The

pre-activation is the same regardless of which activation function is used:

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + b \\ &= (-0.8)(0.6) + (1.5)(0.9) + (-0.3) \\ &= -0.48 + 1.35 - 0.30 \\ &= 0.57 \end{aligned}$$

Sigmoid

$$a = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.57}} = \frac{1}{1.566} \approx 0.639$$

Tanh

$$a = \tanh(z) = \frac{e^{0.57} - e^{-0.57}}{e^{0.57} + e^{-0.57}} = \frac{1.768 - 0.566}{1.768 + 0.566} = \frac{1.202}{2.334} \approx 0.515$$

The approximation errors against the true value $f(0.6, 0.9) = 1.105$ are:

$$\begin{aligned} \varepsilon_{\text{sigmoid}} &= |1.105 - 0.639| = 0.466 \quad (\approx 42\% \text{ relative error}) \\ \varepsilon_{\text{tanh}} &= |1.105 - 0.515| = 0.590 \quad (\approx 53\% \text{ relative error}) \end{aligned}$$

As you can see the error is structural, even if the weights are adjusted it wouldn't yield the desired result. As both of the sigmoid function and tanh function have a range of (0,1) and (-1,1) respectively.

What does this value mean when going back to the original function approximation? Well, for the case of one perceptron it's just a perception point of where the function is at this respective input. However this is not the final step in a Neural network, the

output from a perceptron is then passed through an out put node and is multiplied by its own respective weight v_i , which scales up the value of the input to get a better approximate for the desired function and to extend the range outside that of the activation function.

This process is done at each node in the *hidden layer*. The output is then used for the neural network to make a prediction. Each activation function have their own cases to be used and their respective limitations.

One perceptron is not enough for approximating complex functions, thus we add more for our prediction to become more accurate. This gives rise to the *Multi layer perceptron*, where each layer plays its own role.

2 Multi Layer Perceptrons

A *multi layer perceptron* (MLP) is the foundational architecture of modern neural networks and the building block on which Physics-Informed Neural Networks are constructed.

In the following section we will refer to a perceptron as a *neuron or node* as the name perceptron historically is attributed to usage of the step function which is not in our current focus but is a useful starting point.

2.1 Why are multiple Perceptrons Needed? — The Non-linearity Problem

For simple linearly separable problems, a single perceptron is sufficient. Imagine a dataset of points on a graph, if all points of class A sit on one side and all points of class B on the other, a single straight line separates them perfectly (Figure 5a). The perceptron learns the equation of that line through its weights w_i and bias b .

However, many real-world problems are not linearly separable. Consider a dataset where class A points cluster in two opposite corners of the graph, and class B points cluster in the other two, where no single straight line can separate them (Figure 5b). This is the classical *XOR problem*, used by Minsky and Papert [?] to expose the limits of a single-layer perceptron.

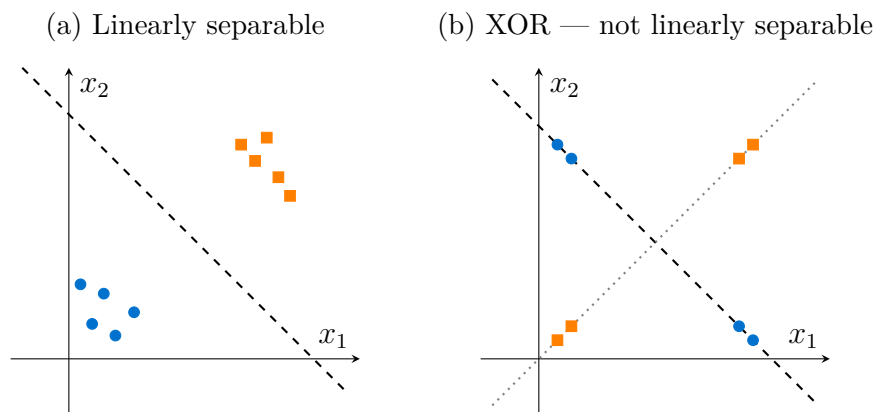


Figure 5: (a) Linearly separable. (b) Non-linearly separable.

To solve for this issue we

Since one perceptron can only produce one bend for each activation function, staking multiples of them in parallel in a layer allows us to create more bends to further better our approximation of the function. Each additional layer allows the network to draw one more *curved* or *composite* decision boundary. With enough neurons and layers, the network can separate arbitrarily complex data distributions — the universal approximation theorem formalises this result [Hornik et al., 1989, ?].

Fig.3 is an example of how perceptron produces only a one bend using the tanh (explained in section 2) and how adding just a couple more gives us more bends to better approximate a function. Without these activation functions we would only have a linear line $wx + b$ and no usefull function.

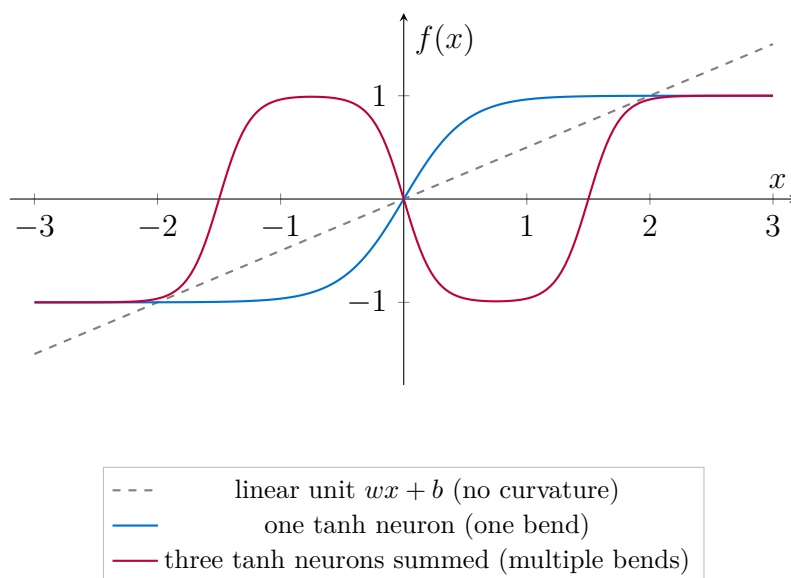


Figure 6: tanh neuron contributes one bend; summing several neurons superposes their bends so the output approximates an increasingly complex curve.

To better show this lets look at 7, where we can see how adding more neurons in a neural network allows it to approximate a simple function such as $f(x) = \sin x$ much more accurately.

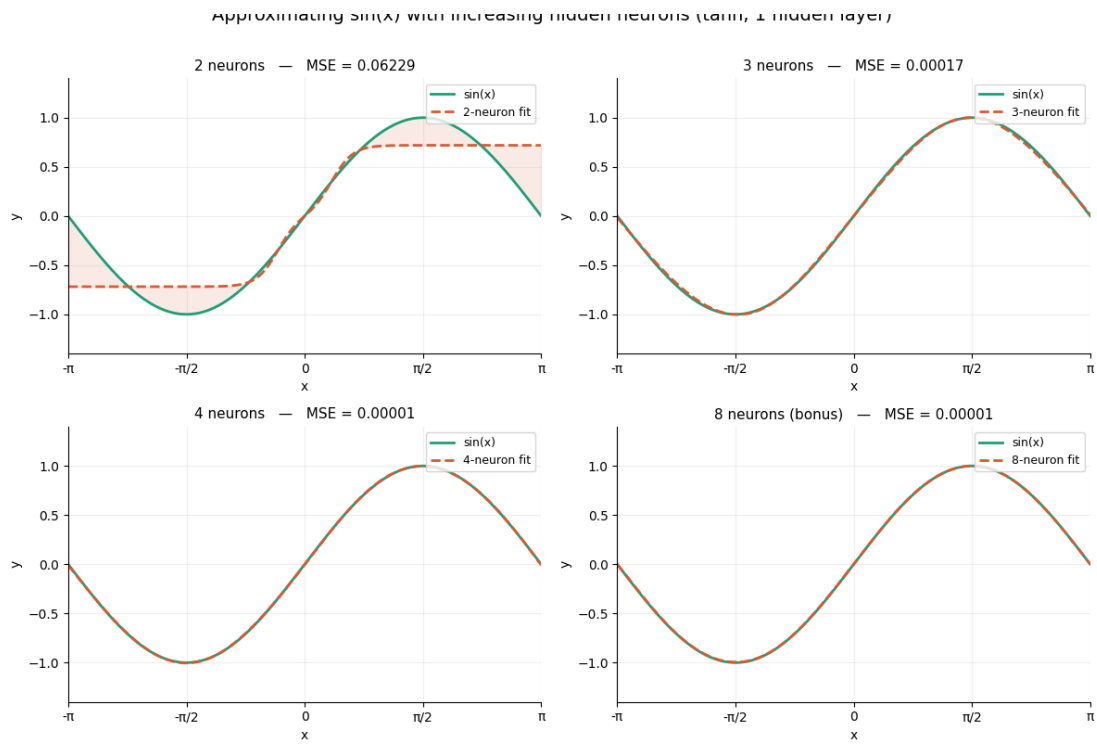


Figure 7: Approximating the Sinx function by adding multiple neurons

2.2 Architecture

An MLP consists of an ordered sequence of *layers*, each containing a set of neurons. Three layer types are distinguished:

- **Input layer:** receives the raw input features.
- **Hidden layers:** one or more intermediate layers that learn internal representations. All neurons in one layer are connected to all neurons in the next.
- **Output layer:** produces the final prediction.

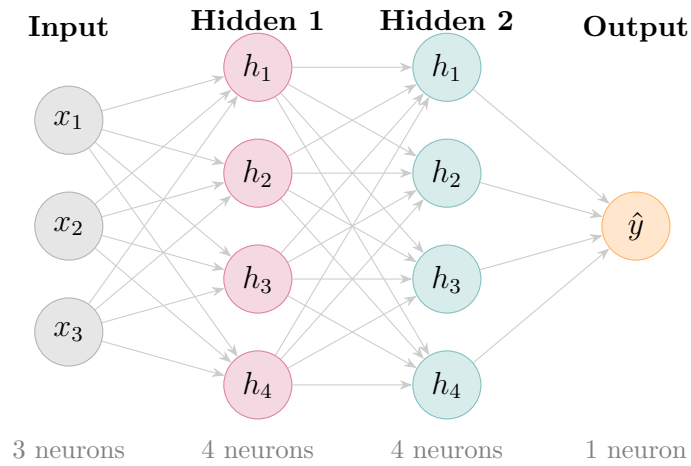


Figure 8: Architecture of an MLP with one input layer (3 neurons), two hidden layers (4 neurons each), and one output layer. Every neuron in a layer connects to every neuron in the next layer.

The *input layer* carries forward the sum of the inputs with their respective weights $w_i x_i$ and the neuron's respective bias b_j .

The *hidden layer* receives the weighted sum value from the previous layer and then passes it through an activation function which introduces the non linearity needed.

In the *output layer* the output of the hidden layer is then multiplied by sizing factor to get a better approximation of the wanted function. Without this layer we would face issues as the one we had with the example of $f(x_1, x_2) = \sin x_1 + x_1 x_2$.

2.3 Why Tanh and Not ReLU or sigmoid for PINNs?

This tutorial is oriented around giving an introduction for PINNs and the basics of neural networks in the context of using for our NiTi wire problem. Therefore moving forward we will start including it more promptly. The PINN loss includes PDE residuals requiring second-order spatial derivatives $\partial^2 f / \partial x^2$ and $\partial^2 f / \partial r^2$. For this to return a meaningful, non-zero value everywhere, the activation must be at least C^2 . ReLU is piecewise linear — its second derivative is zero almost everywhere. Tanh is C^∞ :

$$\sigma(z) = \tanh(z) \tag{3}$$

$$\sigma'(z) = 1 - \tanh^2(z) \tag{4}$$

$$\sigma''(z) = -2 \tanh(z) (1 - \tanh^2(z)) \tag{5}$$

Equation (5) is non-zero at every $z \neq 0$, ensuring that the second derivative of the network output carries meaningful curvature information throughout training.

The reason for using tanh instead of sigmoid for PINNs is a bit more complicated.

- The first is that tanh output are from (-1,1) centered around zero which means they are symmetric and having positive and negative values affects the gradients at a much more higher scale allowing for a much more faster convergence.
- the second is the gradient magnitude, where the derivatives are

$$\begin{aligned} \frac{d}{dz} \tanh(z) &= 1 - \tanh^2(z) \\ \frac{d}{dz} \sigma(z) &= (z)(1 - (z)) \end{aligned}$$

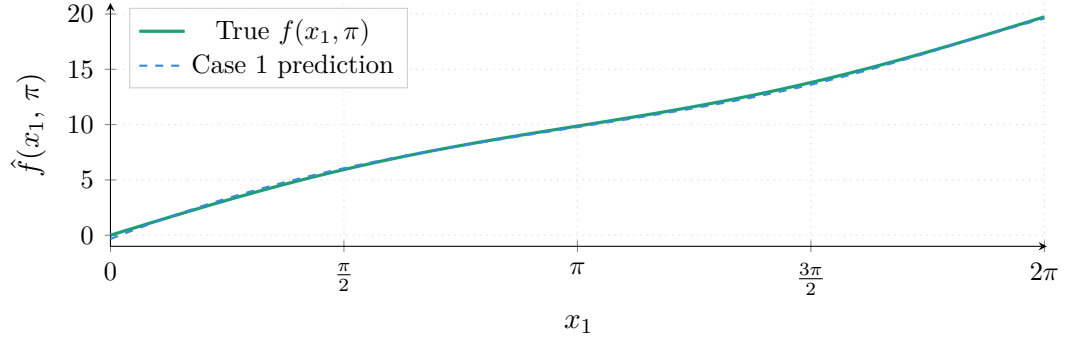
Near zero the gradient for tanh is 1 while its 0.25 for the sigmoid. This plays a great role in backpropagation in neural networks which, where this would cause the gradient of each layer to shrink by a factor of 4. Causing something

know as the *vanishing gradient*.

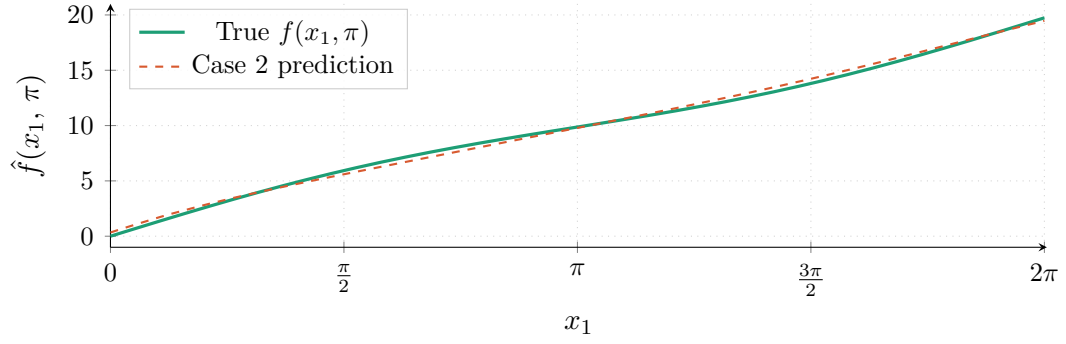
- PDE residual requires also a second derivative. Here again the tanh performs much better due to the derivative giving a much higher values at the ends where this allows the network to learn much faster. This will be shown much better after explain what Backpropagation is.

The question now is how does having an activation function with multiple neurones together give us a better estimate. Lets take a look at our example from before for the function of $f(x_1, x_2) = \sin x_1 + x_1 x_2$. We already saw what the output is for a single neurone, now lets look at a standard Neural network.

(a) Case 1: $[2 \rightarrow 32 \rightarrow 1]$, 129 params, MAE = 0.202



(b) Case 2: $[2 \rightarrow 64 \rightarrow 1]$, 257 params, MAE = 0.335



(c) Case 3: $[2 \rightarrow 32 \rightarrow 32 \rightarrow 1]$, 1185 params, MAE = 0.092

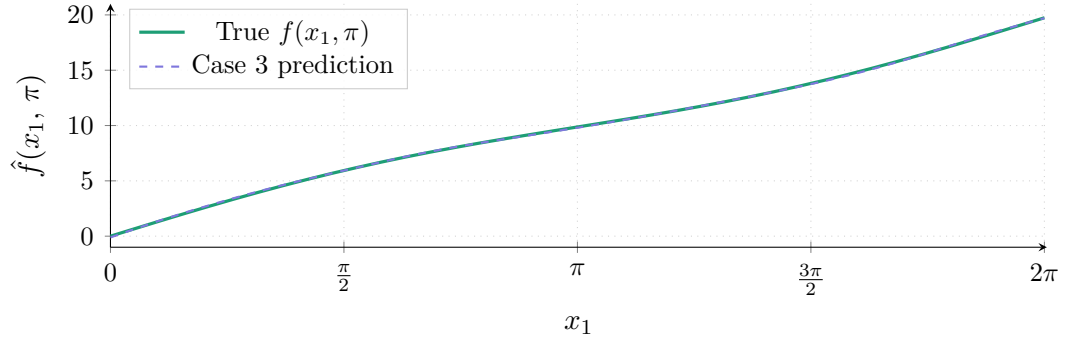


Figure 9: Approximation of $f(x_1, x_2) = \sin(x_1) + x_1 x_2$

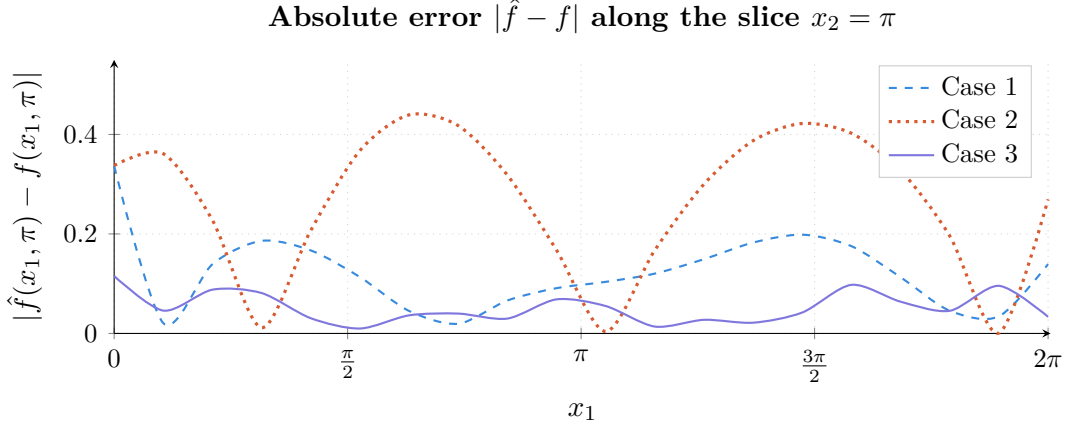


Figure 10: Absolute error for each case

Pointwise absolute error for each architecture along the $x_2 = \pi$ slice. Case 2 (coral dotted) shows the largest and most oscillatory errors. Case 3 (purple solid) consistently achieves the smallest error, showing that adding more layers is more effective than just adding more neurones in a single layer in in this case.

2.4 Fourier Feature Embeddings

2.4.1 Why Does a Standard MLP Struggle with Sharp Gradients?

Standard MLPs exhibit *spectral bias*: gradient descent preferentially fits low-frequency components first and converges slowly on high-frequency spatial variations [Tancik et al., 2020]. For the NiTi wire problem, the sharp thermal gradient near the wire ends — decaying over characteristic length $L_b \approx \sqrt{kR/2h} \approx 9.5$ mm — must be captured faithfully. A plain MLP fits the bulk temperature field accurately but smears this end gradient.

2.4.2 What Are Fourier Feature Embeddings?

The remedy is to transform the raw input coordinates before feeding them to the network [Tancik et al., 2020]:

$$\boldsymbol{\gamma}(\mathbf{x}^*) = \begin{bmatrix} \sin(2\pi \mathbf{B}_1 \cdot \mathbf{x}^*) \\ \cos(2\pi \mathbf{B}_1 \cdot \mathbf{x}^*) \\ \vdots \\ \sin(2\pi \mathbf{B}_m \cdot \mathbf{x}^*) \\ \cos(2\pi \mathbf{B}_m \cdot \mathbf{x}^*) \end{bmatrix} \quad (6)$$

where $\mathbf{x}^* = (x^*, r^*, t^*)$ are the nondimensionalised input coordinates and $\mathbf{B} \in \mathbb{R}^{m \times 3}$ is a fixed random frequency matrix sampled once from $\mathcal{N}(0, \sigma^2)$ before training and held frozen throughout.

$\mathbf{B}$ fixes the frequencies available to the network — analogous to integer harmonics in a classical Fourier series. The MLP weights that follow $\boldsymbol{\gamma}$ learn the amplitudes of those basis functions, playing the role of Fourier coefficients. Because the high-frequency basis functions are explicitly present in the input before training begins, the network is no longer required to generate high-frequency variation from smooth activations. The standard deviation $\sigma \approx 1\text{--}10$ in nondimensional coordinates is typical for thermal field problems.

3 Backpropagation and Automatic Differentiation

Two distinct differentiation processes operate inside a PINN. The first is *backpropagation*, which computes how the loss changes with respect to the network weights θ . The second is *automatic differentiation* (autodiff), which computes how the network output changes with respect to its coordinate inputs (x^*, r^*, t^*) . Both use the chain rule of calculus but ask different questions.

Automatic differentiation (AD) is a technique that is central to machine learning, neural networks, and many other computational fields. AD yields the *exact* derivative of a function – to floating-point precision – regardless of how complex that function is, and it does so at a computational cost proportional to a single forward evaluation of the function.

The two classical alternatives both have significant limitations when executed on a computer. *Finite differences* accumulate round-off error – the error grows as the step size h is reduced and require multiple function evaluations per derivative. *Symbolic differentiation* is exact in principle but suffers from *expression swell*: the size of the symbolic expression grows exponentially with the number of operations, making it impractical for deep networks.¹

AD avoids both problems by applying the chain rule to each elementary operation in the computation graph, accumulating exact numerical derivative values rather than symbolic expressions.

3.0.1 The Chain Rule and Intermediate Variables

AD uses the chain rule applied to composite functions. Consider a scalar function built by composing three operations:

$$y = f(g(h(x))). \quad (7)$$

Introduce intermediate variables $\omega_0, \omega_1, \omega_2, \omega_3$ that name each intermediate quantity:

$$\omega_0 = x, \quad \omega_1 = h(\omega_0), \quad \omega_2 = g(\omega_1), \quad \omega_3 = f(\omega_2) = y. \quad (8)$$

The chain rule then gives the total derivative of y with respect to x as a product of local partial derivatives:

$$\frac{dy}{dx} = \frac{\partial f(\omega_2)}{\partial \omega_2} \cdot \frac{\partial g(\omega_1)}{\partial \omega_1} \cdot \frac{\partial h(\omega_0)}{\partial \omega_0}. \quad (9)$$

At each node in the computation graph, only the *local* partial derivative – the derivative of that node’s output with respect to its own input – is needed. This local quantity is computed exactly and stored numerically; the full derivative is the accumulated product of all such local values along the path from input to output.

¹Baydin et al. ? give a systematic comparison of all three methods; Table 1 in Section 3.3.3 of this document reproduces the key numerical results.

3.0.2 Forward and Reverse Accumulation

Equation (9) can be evaluated in two directions, giving rise to two distinct AD modes.

Forward Accumulation (FA). The product in (9) is evaluated *left to right*, starting from the input x and propagating toward the output y . Define the *tangent* (or *dot*) value of each intermediate variable as its derivative with respect to the chosen input:

$$\dot{\omega}_i = \frac{\partial \omega_i}{\partial x}. \quad (10)$$

The forward sweep propagates tangents according to:

$$\frac{\partial \omega_i}{\partial x} = \frac{\partial \omega_i}{\partial \omega_{i-1}} \cdot \frac{\partial \omega_{i-1}}{\partial x}. \quad (11)$$

The computation begins with a *seed value* $\dot{\omega}_0 = \partial x / \partial x = 1$ and propagates forward through the graph. One forward pass yields the derivative of y with respect to *one* input variable. For a function of n inputs, n forward passes are required to obtain the full gradient.

Backward (Reverse) Accumulation (BA). The product is evaluated *right to left*, starting from the output y and propagating toward the inputs. Define the *adjoint* (or *bar*) value of each intermediate variable as the derivative of the output with respect to that variable:

$$\bar{\omega}_i = \frac{\partial y}{\partial \omega_i}. \quad (12)$$

The reverse sweep propagates adjoints according to:

$$\frac{\partial y}{\partial \omega_i} = \frac{\partial y}{\partial \omega_{i+1}} \cdot \frac{\partial \omega_{i+1}}{\partial \omega_i}. \quad (13)$$

The computation begins with the seed value $\bar{\omega}_3 = \partial y / \partial y = 1$ and propagates backward through the graph. One reverse pass yields the derivative of y with respect to *all* input variables simultaneously.

Key efficiency result. For a scalar output (such as a neural network loss $\mathcal{L}$) and a large number of inputs (such as millions of network weights θ), reverse accumulation computes the full gradient $\nabla_{\theta} \mathcal{L}$ in a single backward pass – at a cost proportional to one forward evaluation. This is precisely why backpropagation, which *is* reverse-mode AD applied to the loss with respect to the weights, is the algorithm that makes deep learning computationally feasible.

The tradeoff is memory: the forward pass values at every node must be stored so that they are available during the backward sweep. For large computation graphs this can require significant memory. Mitigation strategies (gradient checkpointing, reversible architectures) are discussed in the document.

3.1 Worked Example: Forward Accumulation for $f(x_1, x_2) = x_1x_2 + \sin x_1$

Function decomposition

Consider the function

$$y = f(x_1, x_2) = x_1x_2 + \sin x_1. \tag{14}$$

Introduce intermediate variables:

$$\begin{aligned}
\omega_1 &= x_1, \\
\omega_2 &= x_2, \\
\omega_3 &= \omega_1 \omega_2, \\
\omega_4 &= \sin \omega_1, \\
\omega_5 &= \omega_3 + \omega_4 = y.
\end{aligned}$$

Forward pass: computing $\partial y / \partial x_1$

To differentiate with respect to x_1 , initialise the seed values:

$$\dot{\omega}_1 = \frac{\partial \omega_1}{\partial x_1} = 1, \quad \dot{\omega}_2 = \frac{\partial \omega_2}{\partial x_1} = 0. \quad (15)$$

(Setting $\dot{\omega}_2 = 0$ selects x_1 as the independent variable; x_2 is treated as a constant for this pass.) Propagate forward:

$$\dot{\omega}_3 = \dot{\omega}_1 \omega_2 + \omega_1 \dot{\omega}_2 = 1 \cdot \omega_2 + \omega_1 \cdot 0 = \omega_2, \quad (16)$$

$$\dot{\omega}_4 = \cos \omega_1 \cdot \dot{\omega}_1 = \cos \omega_1, \quad (17)$$

$$\dot{\omega}_5 = \dot{\omega}_3 + \dot{\omega}_4 = \omega_2 + \cos \omega_1. \quad (18)$$

Therefore:

$$\frac{\partial y}{\partial x_1} = \omega_2 + \cos \omega_1 = x_2 + \cos x_1. \quad (19)$$

This is the exact result from calculus, obtained by a single forward sweep through the graph with no symbolic manipulation.

Comparison with the computation graph figure

Figure 11 shows the computation graph for this example. In forward accumulation, tangent values flow *upward* (from inputs to output); in reverse accumulation, adjoint values flow *downward* (from output to inputs).

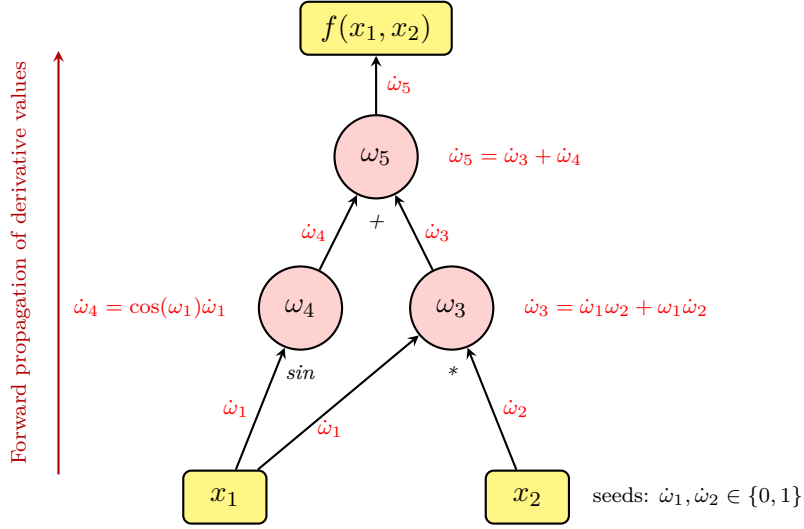


Figure 11: Computation graph for forward accumulation applied to $f(x_1, x_2) = x_1x_2 + \sin x_1$. Pink circles are operation nodes labelled with their primal value ω_i ; yellow boxes are inputs and output. Red edge labels carry the tangent (dot) values $\dot{\omega}_i$ propagated upward from the seed values $\dot{\omega}_1, \dot{\omega}_2 \in \{0, 1\}$.

3.2 In Each Step: Numerical Storage

At each node in the graph, the value of the derivative is stored *numerically* and passed on to the next node. No symbolic expression is constructed. This is the property that keeps the computational cost constant regardless of how many operations compose the function – in sharp contrast to symbolic differentiation, where the expression size grows with each application of the product or chain rule.

3.3 The Computation Graph

Every arithmetic operation a neural network performs is recorded as a node in a *computation graph*. During the **forward pass**, values flow left to right and are

stored at each node. During the **backward pass**, gradients flow right to left: each node multiplies the incoming gradient by its local derivative, accumulating the chain-rule product.

3.4 Worked Example: Two Weights, One Step

Inputs $x_1 = 1$, $x_2 = 2$; weights $w_1 = w_2 = 0.5$; bias $b = 0$; target $y = 4$:

Forward pass:

$$z = 0.5 \times 1 + 0.5 \times 2 = 1.5, \quad \mathcal{L} = (1.5 - 4)^2 = 6.25 \quad (20)$$

Backward pass:

$$\frac{\partial \mathcal{L}}{\partial z} = 2(1.5 - 4) = -5.0 \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = -5.0 \times 1 = -5.0, \quad \frac{\partial \mathcal{L}}{\partial w_2} = -5.0 \times 2 = -10.0 \quad (22)$$

Weight update ($\eta = 0.1$):

$$w_1 \leftarrow 1.0, \quad w_2 \leftarrow 1.5 \quad (23)$$

Verification: $z' = 1.0 \times 1 + 1.5 \times 2 = 4.0$, $\mathcal{L}' = 0$ ✓

w_2 received a gradient twice as large as w_1 because its input $x_2 = 2$ was twice as large as $x_1 = 1$ — input magnitude directly scales the gradient signal a weight receives.

3.4.1 Worked Example: Two Weights, One Step

Consider a two-weight network with inputs $x_1 = 1$, $x_2 = 2$, weights $w_1 = w_2 = 0.5$, bias $b = 0$, and target $y = 4$:

Forward pass:

$$z = w_1x_1 + w_2x_2 + b = 0.5 \times 1 + 0.5 \times 2 + 0 = 1.5 \quad (24)$$

$$\mathcal{L} = (z - y)^2 = (1.5 - 4)^2 = 6.25 \quad (25)$$

Backward pass (applying Eq. ??):

$$\frac{\partial \mathcal{L}}{\partial z} = 2(z - y) = 2(1.5 - 4) = -5.0 \quad (26)$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = -5.0 \times x_1 = -5.0 \times 1 = -5.0 \quad (27)$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w_2} = -5.0 \times x_2 = -5.0 \times 2 = -10.0 \quad (28)$$

Weight update with learning rate $\eta = 0.1$:

$$w_1 \leftarrow 0.5 - 0.1 \times (-5.0) = 1.0 \quad (29)$$

$$w_2 \leftarrow 0.5 - 0.1 \times (-10.0) = 1.5 \quad (30)$$

Verification — new forward pass after one update:

$$z' = 1.0 \times 1 + 1.5 \times 2 = 4.0, \quad \mathcal{L}' = (4.0 - 4)^2 = 0 \quad \checkmark \quad (31)$$

Two observations from this example are important for understanding training behaviour. First, w_2 received a gradient twice as large as w_1 because its input $x_2 = 2$ was twice as large as $x_1 = 1$. Neurons connected to larger inputs receive larger gradient signals and therefore learn faster — the input magnitude directly sets the learning speed for each connection. Second, both gradients are negative because the network predicted too low ($z = 1.5 < y = 4$), so both weights must increase. The sign of the gradient always points in the direction that increases the loss; gradient *descent* moves opposite to this direction.

3.4.2 Example 2 — Sigmoid Neuron

Source: Karpathy (2016), CS231n Lecture Notes [Karpathy \[2016\]](#). Weights $\mathbf{w} = [2, -3, -3]$, inputs $\mathbf{x} = [-1, -2]$.

A sigmoid neuron computes:

$$f(\mathbf{w}, \mathbf{x}) = \sigma(w_0x_0 + w_1x_1 + w_2) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} \quad (32)$$

Forward pass:

$$\text{dot} = w_0x_0 + w_1x_1 + w_2 = (2)(-1) + (-3)(-2) + (-3) = 1 \quad (33)$$

$$e^{-\text{dot}} = e^{-1} \approx 0.3679 \quad (34)$$

$$1 + e^{-\text{dot}} \approx 1.3679 \quad (35)$$

$$\sigma = \frac{1}{1.3679} \approx \mathbf{0.731} \quad (36)$$

Rather than backpropagating through each of the four primitive gates (negate, exponentiate, add one, reciprocal), the sigmoid function has a well-known compact derivative:

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \quad (37)$$

Backward pass (using the grouped sigmoid gradient):

$$\frac{\partial \mathcal{L}}{\partial \text{dot}} = \sigma(1 - \sigma) = 0.731 \times 0.269 \approx \mathbf{0.197} \quad (38)$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = 0.197 \times x_0 = 0.197 \times (-1) = \mathbf{-0.197} \quad (39)$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = 0.197 \times x_1 = 0.197 \times (-2) = \mathbf{-0.394} \quad (40)$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = 0.197 \times 1 = \mathbf{+0.197} \quad (41)$$

Note that w_1 receives exactly twice the gradient magnitude of w_0 because its input

$x_1 = -2$ has twice the magnitude of $x_0 = -1$. This confirms the general principle established in the simpler example: input magnitude directly scales the gradient signal that a weight receives.

The compact grouped derivative eq. (37) is an instance of a broader strategy: any sub-expression whose derivative has a simpler closed form than the chain of its primitive gates can be treated as a single gate. Modern frameworks implement this for all standard activation functions.

3.4.3 Example 3 — Autodiff Exactness: $f(x) = \sin(x) e^x$

The exact derivative of $\sin(x)e^x$ is a standard product-rule exercise. The numerical comparison of autodiff against finite differences is adapted from Baydin et al. [Baydin et al. \[2018\]](#), the survey reference for automatic differentiation in machine learning.

Consider $f(x) = \sin(x) e^x$ evaluated at $x = 1$. Three methods are compared.

Exact derivative (product rule):

$$f'(x) = \cos(x) e^x + \sin(x) e^x = e^x (\sin x + \cos x) \quad (42)$$

$$f'(1) = e^1 (\sin 1 + \cos 1) = 2.71828 \times (0.84147 + 0.54030) = \mathbf{3.75623} \dots \quad (43)$$

Method 1 — Finite differences with $h = 10^{-4}$:

$$\hat{f}'(1) \approx \frac{f(1 + 10^{-4}) - f(1)}{10^{-4}} = \frac{3.75627 \dots - 3.75559 \dots}{10^{-4}} \approx \mathbf{3.75619} \dots \quad (44)$$

$$\text{Error} = |3.75623 - 3.75619| \approx 4.5 \times 10^{-5} \quad (45)$$

Method 2 — Autodiff (computation graph):

The function decomposes into two primitive nodes: $u = \sin(x)$ with $\partial u / \partial x = \cos(x)$, and $v = e^x$ with $\partial v / \partial x = e^x$. Their product $f = uv$ has derivative:

$$\frac{\partial f}{\partial x} = v \cdot \frac{\partial u}{\partial x} + u \cdot \frac{\partial v}{\partial x} = e^x \cos x + \sin x \cdot e^x = e^x (\sin x + \cos x) = \mathbf{3.75623} \dots \quad (46)$$

Error = 0 (to floating-point precision).

Table 1: Comparison of differentiation methods for $f'(1)$ where $f(x) = \sin(x) e^x$.
Exact value: 3.756234...

Method	Result	Error	Cost for $\partial^2 f / \partial x^2$
Finite differences ($h = 10^{-4}$)	3.75619...	$\sim 4.5 \times 10^{-5}$	4 network evaluations, error $\mathcal{O}(h)$
Symbolic differentiation	3.75623...	0	Expression size grows exponentially
Automatic differentiation	3.75623...	0	1 graph traversal, $\mathcal{O}(\text{forward pass})$

For the PINN in this thesis, the PDE residual requires $\partial^2 T^* / \partial x^{*2}$ evaluated at every collocation point. At $N_c = 10,000$ collocation points, finite differences would require $4N_c = 40,000$ network forward passes per training iteration, each introducing $\mathcal{O}(h)$ truncation error that accumulates across the PDE loss. Autodiff computes all N_c second derivatives in a single backward graph traversal at machine precision. This is not a minor optimisation: it is the difference between a computationally feasible and an infeasible training procedure.

3.4.4 Backpropagation vs. Autodiff

Both use the same underlying mechanism — the chain rule — but ask different questions:

- **Backpropagation:** how does the scalar loss $\mathcal{L}$ change with respect to each weight $w_{ij}^{(l)}$? Drives learning.
- **Autodiff for PDE residuals:** how does the network output T^* change with respect to each input coordinate? Produces the PDE residual terms $\partial T^* / \partial x^*$, $\partial^2 T^* / \partial x^{*2}$, etc.

In PyTorch, both are handled by `torch.autograd`. The differentiation variable is

θ for backpropagation, (x^*, r^*, t^*) for autodiff.

3.4.5 How Backpropagation and Autodiff Work Together

During each PINN training iteration:

1. **Forward pass:** evaluate $T^*(x^*, r^*, t^*; \theta)$ at all collocation, boundary, and sensor points.
2. **Autodiff:** compute $\partial T^*/\partial t^*$, $\partial^2 T^*/\partial x^{*2}$, and the radial term to form the PDE residuals res_1 , boundary residuals res_2 , and sensor residuals res_3 .
3. **Loss assembly:**

$$\mathcal{L}(\theta) = \lambda_1 \|\text{res}_1\|^2 + \lambda_2 \|\text{res}_2\|^2 + \lambda_3 \|\text{res}_3\|^2 \quad (47)$$

4. **Backpropagation:** `loss.backward()` computes $\nabla_{\theta} \mathcal{L}$.
5. **Optimiser step:** Adam or L-BFGS updates θ .

Steps 2 and 4 both invoke `torch.autograd` but on different variables.

4 Nondimensionalisation

Before training, every coordinate and physical quantity must be rescaled so that all network inputs and outputs are $\mathcal{O}(1)$.

4.1 Why Nondimensionalisation Matters

The raw dimensional inputs to the NiTi wire PINN differ by up to four orders of magnitude ($x \sim 10^{-2}$ m, $r \sim 10^{-4}$ m, $t \sim 10$ s, $T \sim 300\text{--}400$ K), causing two distinct problems.

Gradient pathology. When inputs span very different scales, the gradients $\partial \mathcal{L} / \partial \theta$ also differ by orders of magnitude across loss terms. The update step is dominated by the large-scale term and the physics in smaller-scale terms is effectively

ignored. Wang et al. [2021] identify this scale mismatch as a primary driver of gradient pathology in PINNs and propose NTK-based adaptive loss weighting as a remedy. Nondimensionalisation removes the root cause before training begins.

Network saturation. The tanh gradient is

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x), \quad (48)$$

which decays to zero as $|x| \rightarrow \infty$. If the raw temperature $T = 293$ K is passed through a tanh layer, $\tanh(293) \approx 1.000$ and the gradient is exactly zero in double precision — the neuron is dead regardless of training steps. After nondimensionalisation, $T^* = 0.5$ gives $\tanh(0.5) \approx 0.462$ with gradient ≈ 0.786 , squarely in the high-gradient active region.

4.2 The General Procedure

Divide each variable by a characteristic scale chosen from the physics:

$$x^* = \frac{x}{L_{\text{ref}}}, \quad t^* = \frac{t}{t_{\text{ref}}}, \quad u^* = \frac{u - u_{\text{ref}}}{\Delta u} \quad (49)$$

- *Spatial*: use the domain size.
- *Temporal*: the diffusion timescale $t_{\text{ref}} = L_{\text{ref}}^2/\alpha$ gives t^* the physical interpretation of the Fourier number.
- *Temperature*: shift by the ambient value and divide by the expected rise $\Delta T = T_{\text{max}} - T_{\infty}$.

Xavier (Glorot) initialisation — the standard for tanh networks — sets weight variance to keep pre-activation values $\mathcal{O}(1)$ when inputs are $\mathcal{O}(1)$. Raw inputs at $\mathcal{O}(10^2)$ or $\mathcal{O}(10^{-4})$ break these assumptions from the first forward pass.

4.3 Worked Example: the 1D Transient Heat Equation

The 1D transient heat equation in a rod of length L :

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} \quad (50)$$

Apply the substitutions $x^* = x/L$, $t^* = \alpha t/L^2$, $T^* = (T - T_\infty)/\Delta T$ with $\alpha = k/(\rho c_p)$. Since $\rho c_p \alpha = k$ by definition, the factor $k\Delta T/L^2$ cancels from both sides, leaving:

$$\boxed{\frac{\partial T^*}{\partial t^*} = \frac{\partial^2 T^*}{\partial x^{*2}}} \quad (51)$$

No dimensionless groups appear. The diffusion timescale absorbs all material properties exactly; once nondimensionalised, the PINN sees the same equation for steel, copper, or NiTi, with material information encoded only in the boundary and initial conditions.

4.4 Application to the NiTi Wire Problem

The 2D axisymmetric transient heat equation is:

$$\rho c_p \frac{\partial T}{\partial t} = k \left[\frac{\partial^2 T}{\partial x^2} + \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right) \right] + Q \quad (52)$$

Following [Cai et al. \[2021\]](#):

$$x^* = \frac{x}{L_w}, \quad r^* = \frac{r}{R}, \quad t^* = \frac{\alpha t}{L_w^2}, \quad T^* = \frac{T - T_\infty}{\Delta T} \quad (53)$$

Note: x^* is scaled by L_w and r^* by R . Both map to $[0, 1]$ but represent physical lengths differing by $A = L_w/R \approx 492$.

After substitution and division by $k\Delta T/L_w^2$:

$$\boxed{\frac{\partial T^*}{\partial t^*} = \frac{\partial^2 T^*}{\partial x^{*2}} + A^2 \cdot \frac{1}{r^*} \frac{\partial}{\partial r^*} \left(r^* \frac{\partial T^*}{\partial r^*} \right) + Q^*} \quad (54)$$

with dimensionless parameters:

$$A = \frac{L_w}{R} \quad (\text{wire aspect ratio}), \quad (55)$$

$$Q^* = \frac{Q L_w^2}{k \Delta T} \quad (\text{dimensionless Joule heating}). \quad (56)$$

Boundary and initial conditions.

$$T^*(x^*, r^*, 0) = 0 \quad (57)$$

$$-\frac{\partial T^*}{\partial r^*} \Big|_{r^*=1} = \text{Bi} \cdot T^*, \quad \text{Bi} = \frac{hR}{k} \quad (58)$$

$$T^*(0, r^*, t^*) = T^*(1, r^*, t^*) = 0 \quad (59)$$

$$\frac{\partial T^*}{\partial r^*} \Big|_{r^*=0} = 0 \quad (60)$$

Numerical values for Flexinol HT 0.012". $k = 18 \text{ W m}^{-1}\text{K}^{-1}$, $\rho = 6450 \text{ kg m}^{-3}$, $c_p = 490 \text{ J kg}^{-1}\text{K}^{-1}$, $R = 1.525 \times 10^{-4} \text{ m}$, $L_w = 0.075 \text{ m}$:

$$\alpha \approx 5.70 \times 10^{-6} \text{ m}^2\text{s}^{-1}, \quad (61)$$

$$A \approx 492, \quad A^2 \approx 2.42 \times 10^5, \quad (62)$$

$$\text{Bi}(h = 10) \approx 8.5 \times 10^{-5}, \quad \text{Bi}(h = 25) \approx 2.1 \times 10^{-4}. \quad (63)$$

$\text{Bi} \ll 1$ confirms the cross-section is nearly isothermal: radial temperature gradients are negligible in the physical domain. The full 2D cylindrical formulation is retained in the PINN to maintain physical consistency with the FeniCS results.

For $t_f = 30 \text{ s}$ the dimensionless end time is:

$$t_f^* = \frac{\alpha t_f}{L_w^2} \approx 0.030. \quad (64)$$

The experiment captures early-time transient behaviour ($\text{Fo} \ll 1$). $t^* \in [0, 0.030]$ remains well within the active region of $\tanh$.

4.5 What the Network Sees Before and After Scaling

Table 2: Network inputs before and after nondimensionalisation.

Variable	Without scaling	With scaling	Comment
Axial position	$x \in [0, 0.075]$ m	$x^* \in [0, 1]$	Normalised by wire length
Radial position	$r \in [0, 1.525 \times 10^{-4}]$ m	$r^* \in [0, 1]$	$\approx 500\times$ smaller than x without scaling
Time	$t \in [0, 30]$ s	$t^* \in [0, 0.030]$	Fourier number; experiment is early transient
Temperature	$T \in [293, 393]$ K	$T^* \in [0, 1]$	Saturates tanh completely without scaling

4.6 The Three Dimensionless Groups

Table 3: Dimensionless groups of the NiTi wire thermal problem. $L_w = 75$ mm, $h = 10 \text{ W m}^{-2}\text{K}^{-1}$.

Group	Definition	Physical meaning	Value	Regime
A	L_w/R	Wire aspect ratio; governs relative axial vs. radial coordinate scale	492	$A \gg 1$: slender wire
Bi	hR/k	Convective resistance / conductive resistance	8.5×10^{-5}	$Bi \ll 1$: isothermal cross-section
Q^*	$QL_w^2/(k\Delta T)$	Heat generation rate / axial conductive removal rate	depends on $I, \Delta T$	$Q^* \sim 1$: balanced

A^2 — **not a physical amplification.** One unit of r^* corresponds to a physical distance of only $R = 0.1525$ mm, while one unit of x^* corresponds to $L_w = 75$ mm. Each differentiation brings a factor of $1/\ell$, so the ratio between radial and axial second-derivative terms in physical space appears as A^2 in the dimensionless equation. The prefactor restores the correct relative magnitude; without it, the dimensionless PDE would be wrong.

$Bi \ll 1$ and $A^2 \gg 1$ are simultaneously true and consistent: the radial gradient is tiny in physical space because the cross-section is nearly isothermal, yet that tiny gradient expressed per unit of r^* is amplified by A because r^* is a compressed coordinate.

References

- A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- S. Cai, Z. Wang, S. Wang, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks for heat transfer problems. *ASME Journal of Heat Transfer*, 143(6):060801, 2021. doi: 10.1115/1.4050542.
- K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.
- A. Karpathy. CS231n: Convolutional neural networks for visual recognition — lecture notes on backpropagation. Stanford University, 2016. URL <https://cs231n.github.io/optimization-2/>.
- M. Tancik, P. P. Srinivasan, B. Mildenhall, et al. Fourier features let networks learn high-frequency functions in low-dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- S. Wang, Y. Teng, and P. Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43:A3055–A3081, 2021. doi: 10.1137/20M1318043.